Toward Latency-Sensitive Generative AI Provision via Dynamic Utility Maximization in Serverless Mobile Cloud-Edge Networks

Lianbo Ma, *Senior Member, IEEE*, Jiacheng Ding, Ying Qian, Qiang He, Yuanguo Bi, and Qing Li

***Abstract*—Generative AI (GenAI) has become a research hotspot for the task of content creation and production, which suffers from the issue of high latency due to cloud transmission. One effective solution is to integrate serverless computing with mobile edge computing (MEC) to build a communication-efficient GenAI system, where serverless functions are executed via containers on edge servers. However, the nonnegligible latency of container deployment and cold starts degrades the quality of GenAI service. This issue becomes even more serious in dynamic MEC with mobile and uncertain users. In this paper, we study the provisioning of latency-sensitive query services in GenAI-enabled serverless MEC through dynamic utility maximization. While GenAI of users deployed in cloud refers to as primary GenAI, we deploy their GenAI replicas based on serverless functions in edge servers to maximize user service satisfaction (i.e., utility function). We first formulate a joint decision problem, i.e., GenAI Replica Allocation and Placement (GRAP) problem, under various resource constraints. For this problem, we propose an approximation solver with a provable approximation ratio. Then, we consider an dynamic GRAP problem with uncertain values of users and stochastic request arrivals, and devise a performance-guaranteed online algorithm for a special case of the problem by assuming only a small subset of edge servers suffers significant utility degradation. Finally, theoretical analysis and experiments validate the effectiveness of the proposed mechanisms, which consistently outperform baseline methods in both service latency and user satisfaction.**

***Index Terms*—Generative AI, mobile edge computing, resource allocation, latency-sensitive service, utility maximization.**

I. Introduction

GENERATIVE AI (GenAI), also called AI-generated content (AIGC), has become a transformative technique to provide advanced creative content generation services through imitating human creative content [1], [2], [3], which has enabled various new AI applications, such as text-to-image generation [4], chatbot [5], and AI-enhanced mixed reality (MR) [6], [7]. Given high complexity of target generative tasks, the implementation of GenAI relies on large-scale AI models, e.g., ChatGPT [10] and DeepSeek [11], necessitating vast storage and computing power (e.g., thousands of GPUs) for both training and inference processes. GenAI models are usually trained and run in cloud servers, allowing users to submit frequent content-generation requests through cloud network, which however suffers from high latency and increased backhaul traffic load. Then, mobile edge computing (MEC) is utilized for cloud-edge collaboration, which facilitates access to real-time GenAI services with ultra-low latency and reduced bandwidth consumption [12], [13], [14], [29], [33].

A. Motivation

However, cloudlets deployed at edge servers still struggle with limited edge resources and poor GenAI management [1], [18], [34]. Accordingly, serverless computing has emerged as a potential way to deliver fine-grained and highly-elastic resource allocation towards serverless cloud-edge collaboration by utilizing container-based serverless functions [19], [20], [22]. In serverless cloud-edge networks, GenAI service functions can be dynamically activated and terminated in edge servers with container packaging technique, significantly reducing over-provisioning and resource wastage.

In this paper, we explore the provision of latency-sensitive query services of GenAI models in serverless cloud-edge networks using containerization technique. In cloud network, each user sends his task request to a primary GenAI model built in remote cloud server for target task. By executing the primary GenAI models collaboratively in cloud server, each user can receive user-centric query results from the cloud, but long delays occur due to user-cloud communication [12]. To mitigate this, we deploy GenAI replicas of the primary GenAI models in edge servers, with each replica implemented as a serverless function for efficient management. To avoid the long service delays associated with querying the primary GenAI, users can offload task requests to a GenAI replica deployed in edge server, achieving query results with minimal latency. If the GenAI replica in edge server cannot satisfy the user requests, it will be updated (redeployed) by the suitable primary GenAI model from cloud server.

Received 24 October 2025; revised 14 February 2026; accepted 31 March 2026. Date of publication 6 April 2026; date of current version 12 June 2026. This work was supported by the National Natural Science Foundation of China under Grant 62472079. *(Corresponding author: Qiang He.)*

Lianbo Ma, Jiacheng Ding, and Ying Qian are with the College of Software, Northeastern University, Shenyang 110819, China (e-mail: malb@swc.neu.edu.cn; 2371440@stu.mail.neu.edu.cn; 2110494@stu.mail.neu.edu.cn).

Qiang He and Yuanguo Bi are with the College of Computer Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: heqiang@mail.neu.edu.cn; biyuanguo@ise.neu.edu.cn).

Qing Li is with Peng Cheng Laboratory, Shenzhen 518055, China (e-mail: liq@pcl.ac.cn).

Digital Object Identifier 10.1109/TSC.2026.3680952

1939-1374 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

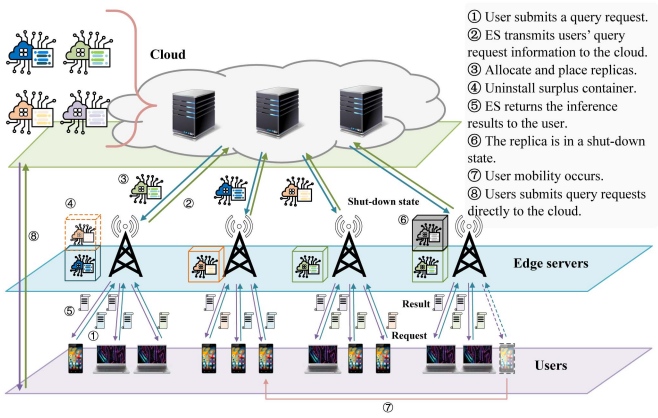


Fig. 1. System architecture diagram.

B. Challenges and Novelty

However, deploying GenAI replicas in edge servers to offer low-latency query services for mobile users still faces several key challenges.

Challenge 1. GenAI replica allocation under heterogeneous resource limitations: Due to the heterogeneous nature of cloud-edge resources in serverless MEC, it is nontrivial to allocate the candidate GenAI replicas to suitable edge servers (with different resource configures) such that maximal user utility can be achieved. In serverless MEC, edge servers exhibit resource heterogeneity with different resource limitations [13], while GenAI models and user requests also demonstrate significant variability in resource consumption and service delays [1]. Factors such as CPU and memory requirements, transmission and inference delays further complicate the allocation of GenAI replicas, necessitating a tailored task allocation to ensure optimal deployment.

*Challenge 2. Considerable latency of GenAI containers' cold start:*In serverless computing, numerous initialization operations required to instantiate a container result in significant cold start latency, adversely affecting the user service satisfaction, and the typical solution is to warm up existing containers by reusing them, largely reducing the start-up latency. However, at each service interval, many users apply for GenAI services, raising a critical question: how to place GenAI replicas in edge servers to maximize the utility gain of query services, by instantiating new containers or reusing existing ones? This placement problem becomes increasingly complex when accounting for the limitation and the heterogeneity in resource availability and capabilities.

Challenge 3. Utility gain maximization over uncertain user information and stochastic request arrivals: Much information about the serverless cloud-edge system (with dynamic query requests from mobile uses) is uncertain. The request preferences for GenAI services (referred to as the values of user) exhibit high variability and uncertainty (e.g., service types, sizes and latencies), and it is difficult to capture the prior knowledge of future request arrivals. The above uncertainty and stochasticity may lead to frequent fallbacks to the cloud for GenAI inference services and repeated cold starts of containers in edge servers, which further exacerbates the service latency and resource waste.

To address the above challenges, we study the provisioning of latency-sensitive query services in GenAI-enabled serverless cloud-edge networks through dynamic utility maximization. The novelty of our design lies in the dynamic allocation and placement of GenAI replicas under conditions of high user mobility, uncertain user query information and various resource constraints. Also, we apply GRAP modeling to describe user satisfaction with GenAI query services. We design an approximation algorithm and an online algorithm with performance guarantees to solve the target problems.

To sum up, the main contributions of this work include:

Contribution 1: We propose two novel joint decision problems: 1) the **G**enAI **R**eplica **A**llocation and **P**lacement (**GRAP**) problem under storage and computing resource constraints (**Challenge 1**) and 2) the online GenAI replica allocation and placement (**dynamic GRAP**) problem addressing uncertain query patterns from highly mobile users (**Challenge 3**), where a comprehensive utility indicator is designed to quantify user service satisfaction.

Contribution 2: We propose an approximation algorithm with a provable approximation ratio to solve the GRAP problem, which leads to an Integer Linear Programming (ILP) solution that jointly determines: 1) the allocation of containers from cloud to ESs according user requests and ES resources and 2) the placement of containers in ESs (e.g., instantiating new containers or reusing existing ones), which reduces cold-start latency while maximizing utility gain (**Challenge 2**).

Contribution 3: We propose two online algorithms to solve dynamic GRAP. Especially, we exploit real-time system state and historical usage to assess dynamic cold-start latency and update overhead, and integrate current queries and system status to achieve optimal allocation and deployment solution, without need of prior knowledge of future query requests.

Contribution 4: We carry out theoretical analysis and extensive experimentation through simulation and a real-world testbed to validate the effectiveness of the proposed designs, outperforming the counterparts. Results show that the proposed methods deliver up to 44.3% lower service latency than those of baseline methods.

II. System Model and Problem Definitions

A. System Model

We consider a mobile cloud-edge network $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, as shown in Fig. 1, where V_{ES} is a set of edge servers (ESs), V_{MD} is a set of mobile devices (MD)/users, L_{ed} is the set of links connecting these ESs, and v_{sc} is a cloud server. Each ES is located with an access point, while each MD acts as user with high mobility, and all ESs are interconnected by an optical fiber network with imperceptible communication delay. Each GenAI service is offered through a serverless container, where the key resource demand is memory resource, as both data processing capacity and computational needs scale proportionally with allocated container memory. Let m_e be the available memory capacity of ES $e \in V_{ES}$, and D_e

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

denote the communication delay of transmitting one unit of data along link L_{ed}. Each ES $e \in V_{ES}$ connects to the remote cloud v_{sc} through a gateway, creating significantly higher end-to-end delay than the communication between any two ES nodes [12], [46], [47]. The system operation is discretely divided into a set of checkpoints (i.e., $p = 1, 2, \ldots, |P|$). The system's execution flow is shown in Fig. 1 (① - ⑧).

Cloud-edge severless computing: The roles of the main components in serverless computing [1] are as below.

1) *Cloud:* Serving as the central management hub, the cloud v_{sc} maintains a repository of GenAI models generated from various data sources, together with user query records collected through ESs, and periodically updates the placement of GenAI replicas across ESs. Also, the cloud can directly handle user queries to provide requested services when necessary.
2) *Edge Servers (ES):* As the bridge between cloud and users, each ES e hosts a replica of the GenAI model in a container to enable direct query processing at the edge-bypassing the cloud and ensuring low-latency responses. Assume that each ES, with limited resources (e.g., storage and bandwidth), covers $\widehat{u}$ users and can deploy up to m kinds of container.
3) *Edge Devices (MDs):* Each MD $u \in V_{MD}$ submits a GenAI query request q_u, which is characterized by a tuple (I_u, O_u), where I_u denotes the length of the input context (prompt tokens) and O_u represents the expected length of the generated sequence (output tokens). The total context window size is $L_{seq} = I_u + O_u$. For each ES e, let $Q_e = \{q_{e1}, \ldots, q_{e\widehat{u}}\}$ be the set of query requests from all users within its coverage. In fact, users are inherently mobile and may move frequently between ESs during query.

Containerized GenAI service: The GenAI model of each user $u \in V_{MD}$ is deployed in the remote cloud v_{sc}, termed as *the primary GenAI model*. While user queries can be processed through primary GenAI model in cloud, it results in unacceptable latency. For this issue, these user queries can be proceeded through containerized GenAI service: 1) *the GenAI replica* deployed at ES handles delay-sensitive queries by processing raw data locally and returning results directly to the user, and 2) the system simultaneously compresses some user data (e.g., collected user requests, response time, error logs) at the edge and transmits it to the primary model in cloud for synchronization.

In principle, this cloud-edge serverless computing enables flexible query processing at either the edge replica or cloud primary model, optimizing both performance and resource utilization of target GenAI service.

B. GenAI Queries With Mobility and Resource Constrains

For each MD u, each GenAI replica is implemented as a containerized serverless function, consuming a amount mum_u of memory resources. Instantiating a container incurs a significant cold start latency. For containers with identical packages, we can utilize warm start to eliminate the latency of container instantiation, i.e., a container created for user u at checkpoint p can be reused for subsequent queries at $t+1$. Notably, the primary GenAI models in cloud remain perpetually warm due to the cloud's abundant computational resources, ensuring constant readiness for service requests. Moreover, users are allowed to move across ESs, issuing queries at each checkpoint without any knowledge on future user queries.

Storage and computing resource limitations: For each ES e, the storage resource is limited by the size s_e; the computing power (for implementation of containers) is denoted as C_e; the bandwidth is limited by b_e. For each container c, we define m_c as the required storage space and C_c as the computing resources needed during a warm start. Unlike traditional workloads, the memory footprint of GenAI inference is dynamic, dominated by the Key-Value (KV) cache which grows linearly with the sequence length. We model the memory consumption $m_c(q_u)$ for processing request q_u on edge server e as: $m_c(q_u) = \Psi_{model} + 2 \cdot L \cdot H \cdot (I_u + O_u) \cdot \beta_{prec} \cdot B_{size}$, where Ψ_{model} is the static VRAM usage for model weights, L and H denote the number of layers and hidden size of the LLM, β_{prec} is the precision, and B_{size} is the batch size.

And then, two state variables, $x_{e,c}$ and $h_{e,c}$, are used to represent the deployment status of the container on each ES at the current checkpoint, i.e.,

$$x_{e,c} = \begin{cases} 0 & \text{container } c \text{ is not deployed on ES } e, \\ 1 & \text{container } c \text{ is deployed on ES } e, \end{cases} \tag{1}$$

where $x_{e,c} = 1$ indicates that at the observation checkpoint, and c is deployed in e, while also having:

$$h_{e,c} = \begin{cases} 0 & \text{container } c \text{ is not started on ES } e, \\ 1 & \text{container } c \text{ is warm started on ES } e, \end{cases} \tag{2}$$

where $h_{e,c} = 1$ means that at the observation checkpoint, container c is deployed in e at a warm start state. Then, based on above constrains, we derive the total storage and computing constraints of each ES as follows:

$$\sum_{e=1}^{k} \sum_{c=1}^{k} m_c x_{e,c} \leq m_e, \tag{3}$$

$$\sum_{e=1}^{k} \sum_{c=1}^{k} C_c x_{e,c} h_{e,c} \leq C_e. \tag{4}$$

C. Latency Model

The service latency of an user query includes 1) the communication delay between user and GenAI, 2) the processing latency of GenAI, and 3) the latency of container deployment from cloud to ESs.

The communication delay between user and GenAI: If a user u moves to an ES e at checkpoint p and issues a query with raw data size $I_u + O_u$.

[1] Despite its name, serverless computing [19] typically relies on a centralized server, which is however managed entirely by cloud, enabling developers to focus solely on application logic without direct server intervention.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I
KEY NOTATIONS IN THIS PAPER

Notation	Description
m_e, m_c	Memory constraints for edge server e and container c.
b_e, b_{sc}	Available bandwidth of ESr e and the bandwidth between user and cloud.
C_e, C_c	Computing capacity of ES e and the computing demand of container c.
$\mathcal{H}(u,e)$,τ_i	Hop set from user u to ES e; per-hop propagation/queuing constant.
set^a, set^p	The allocation set; the placement set.
$\overline{U}_1$, $\overline{U}_2$, $\overline{U}_3$, $\widehat{U}$, $\widehat{U}_1$, $\widehat{U}_2$	The sets of users affected by different policies during the strategy adjustment process.
$s(u,t)$, s_c	The data stream from user u at checkpoint p, and the size of container c
$x_{e,c}$, $h_{e,c}$	Indicator for deploying container c in ES e, indicator for warm-starting container c in ES e.
β	A weighting coefficient for checkpoint.
Freq	Historical usage frequency (e.g., number of times a container is requested).
ξ	A balancing factor between different utility terms.
λ_1, λ_2	Penalty/weighting parameters for historical usage in container placement.
α	Tuning parameter for container allocation and placement in dynamic GRAP problems.

Case 1: If the requested GenAI replica is available in ES e, the user can directly transmit the query data to e without invoking the cloud. Then, the corresponding communication delay $D_{\text{toE}}(u,e)$, i.e., the total time required to deliver data from u to e, can be defined as:

$$D_{\text{toE}}(u,e) = \sum_{i\in\mathcal{H}(u,e)} \left(\frac{I_u}{b_e^{pre}} + \frac{O_u}{b_e^{dec}}\right) + \sum_{i\in\mathcal{H}(u,e)} \tau_i, \quad (5)$$

where $\mathcal{H}(u,e)$ is the set of hops on the route from u to e, and bandwidth b_e^{pre} and b_e^{dec} represent the token processing throughput (tokens/s) of edge server e for the prefill and decoding phases, and τ_i is queuing constant.

Case 2: If no GenAI replica is available in ES e, or if the available container is at the shut-down state, the user request should be redirected to the cloud for processing. Then, the communication delay D_{toC} from ES e to cloud v_{sc}) through the backbone path $P(e,v_{sc})$, can be formulated as:

$$D_{\text{toC}}(u) = \sum_{i\in\mathcal{H}(e,v_{sc})} \left(\frac{I_u}{b_e^{pre}} + \frac{O_u}{b_e^{dec}}\right) + \sum_{i\in\mathcal{H}(e,v_{sc})} \tau_i. \quad (6)$$

We assume that the set of users submitting query requests at a given checkpoint is $\widehat{U} \in U$, which is divided into two subsets: $\widehat{U}_1$ accesses services through containers in ES, and $\widehat{U}_2$ accesses services directly from the cloud. Thus, the total service delay for all users in the system can be expressed as:

$$D_{con} = \sum_{u\in\widehat{U}_1} D_{toE}(u,e) + \sum_{u\in\widehat{U}_2} D_{toC}(u,e). \quad (7)$$

The processing latency of GenAI: Let $\sigma_{u,v_{sc}}$ be the processing time of a unit of data by the primary GenAI of user u in remote cloud v_{sc}, and $\sigma_{u,e}$ be the processing time of a unit of data by the GenAI replica of user u in ES e. We analyze the processing delay of the GenAI from the following two cases.

Case 1: The query of user u is processed by the GenAI replica in an ES $e \in V$, whose container is instantiated in a cold start manner. Let D_{cld} be the cold start delay of instantiating the container in ES $e \in V$ at checkpoint p, which encapsulates the GenAI replica of user u. Then, the processing delay of the GenAI replica of user u in $e \in V$ at checkpoint p is defined as:

$$D_{prc}(u,v) = D_{cld} + \sigma_{u,e} \cdot \left(\frac{I_u}{b_e^{pre}} + \frac{O_u}{b_e^{dec}}\right). \quad (8)$$

Case 2: The query of user u is processed by the primary GenAI in remote cloud v_{sc}, or the GenAI replica in ES e in a warm-start manner, i.e., the user u reuses the existing container of the GenAI replica in e. Then, the processing delay of the primary GenAI of user u in node $v \in V_{ES} \cup v_{sc}$ at checkpoint p is calculated as:

$$D_{prc}(u,v) = \sigma_{u,v} \cdot \left(\frac{I_u}{b_e^{pre}} + \frac{O_u}{b_e^{dec}}\right). \quad (9)$$

The latency of container deployment: For some query requests, the ESs without effective GenAI replicas have to redeploy new containers allocated from cloud, which results in significant latency overhead. We analyze this deployment delay of GenAI containers from the following two cases.

Case 1: When sufficient free space is available on target ES, the allocated container is directly transferred from cloud to target ES and then installed and instantiated in a cold start way. In this process, the incurred transfer delay is defined as:

$$D_{tran}(e,c) = \frac{s_c}{b_e}. \quad (10)$$

Considering the delay D_{ins} of installing container on ES and the cold start delay D_{cld}, we can obtain the deployment delay of this case:

$$D_{dep}^1 = D_{tran} + D_{ins} + D_{cld}. \quad (11)$$

Case 2: When no free space is available on the ES, the target ES first performs a selective free-up operation to reclaim sufficient resources by evicting a specific subset of containers determined by the allocation algorithm (to maximize system utility). This operation incurs a processing delay D_{uins}, which relates specifically to the size of the evicted containers rather than the entire server capacity. Thus, the allocation delay of this case is defined as:

$$D_{dep}^2 = D_{uins} + D_{tran} + D_{ins} + D_{cld}. \quad (12)$$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

Based on above two cases, we can formulate the deployment delay generated during the container allocation stage as:

$$D_{rd} = \sum_{q_u \in \overline{U}_1} D_{cld} + \sum_{q_u \in \overline{U}_2} D^1_{dep} + \sum_{q_u \in \overline{U}_3} D^2_{dep}, \tag{13}$$

where, $\overline{U}_1$ represents the task (query request) set of users in scenario where target ESs do not require new allocation but instantiate their containers in a cold start way, i.e., $x_{e,c} = 1$ and $h_{e,c} = 0$; $\overline{U}_2$ ($\overline{U}_3$) denotes the task set of users in scenario where target ESs have (have no) free space to redeploy new containers allocated from cloud, i.e., $x_{e,c} = 0$ and $h_{e,c} = 0$.

The queueing latency: Let $\lambda_e(p)$ be the arrival rate of queries observed at ES e during checkpoint p and let $\mu_e(p)$ denote the effective service rate determined by the deployed replicas and their scaling levels. Assuming an M/M/1 queue with $\rho_e(p) = \lambda_e(p)/\mu_e(p) < 1$, the mean waiting time is

$$D_{\mathrm{q}}(u, e, p) = \frac{1}{\mu_e(p) - \lambda_e(p)}. \tag{14}$$

D. Utility Defintion

Queries can be processed by the primary GenAI in remote cloud v_{sc}, which however results in high query service delays. By deploying GenAI replicas on ESs for query processing, these delays can be significantly reduced, enhancing user satisfaction. For each user $u \in V_{MD}$ submitting a query, we define the utility to depict the user satisfaction degree from the following two aspects.

Part 1: System latency: Given query request q_u of user u, which is proceeded by the GenAI container c in ES e, the resultant system latency is formulated as:

$$\begin{aligned} D_{q_u} = {} & \tilde{x}_{e,c}\tilde{h}_{e,c}D_{toE}(u,e) + \tilde{x}_{e,c}(1-\tilde{h}_{e,c})(D_{toE}(u,e) \\ & + D_{cld}) + (1-\tilde{x}_{e,c})D_{toC}(u) + D_{prc}(u,v) \\ & + \tilde{x}_{e,c}\tilde{h}_{e,c}\, D_{\mathrm{q}}(u,e,p), \end{aligned} \tag{15}$$

where $\tilde{x}_{e,c}$ denotes that container j is deployed in ES e, and $\tilde{h}_{e,c}$ represents that container c is instantiate in e, i.e.,

$$\tilde{x}_{e,c} = \begin{cases} 1 & \text{container } c \text{ will be deployed in ES } e, \\ 0 & \text{container } c \text{ will not be deployed in ES } e. \end{cases} \tag{16}$$

$$\tilde{h}_{e,c} = \begin{cases} 1 & \text{container } c \text{ will be instantiated in ES } e, \\ 0 & \text{container } c \text{ will not be instantiated in ES } e. \end{cases} \tag{17}$$

Part 2: Historical usage preference: The historical data from user query requests in the cloud is utilized to model historical usage preferences for containers. We formulate a reference value $Freq_{e,c}$ based on the historical usage of each container c in each ES e, capturing its past utilization, defined as:

$$Freq_{e,c} = \sum_{t=1} r_{e,c}, Freq_{e,c} \in [0, upper], \tag{18}$$

where $upper$ denotes the upper bound values, determined based on whether or not the user encourages learning about the historical usage pattern, and the Boolean value $r_{e,c}$ indicates whether the container is accessed within a given checkpoint, defined as:

$$r_{e,c} = \begin{cases} 1 & \text{container } c \text{ is used in ES } e, \\ -1 & \text{container } c \text{ is not used in ES } e. \end{cases} \tag{19}$$

We naturally prefer to place containers with higher $Freq_{e,c}$ values on their corresponding ES. Accordingly, this historical usage preference is formulated as a regularization function:

$$Reg_{e,c} = \left[\lambda_1(1-\tilde{x}_{e,c}) + \lambda_2(1-\tilde{h}_{e,c})\tilde{x}_{e,c}\right] Freq_{e,c}, \tag{20}$$

where λ_1 and λ_2 are the coefficients.

In fact, the proposed utility function is derived from the Quality of Experience (QoE) theory [36] widely adopted in latency-sensitive service optimization. In particular, the user satisfaction can be regarded as a decreasing convex function of the perceived latency and an increasing function of service familiarity or preference. Accordingly, our utility model combines two components: 1) a latency-related term that penalizes long response times (reflecting TTFT and generation rate requirements), and 2) a historical usage preference term $Freq_{e,c}$ that captures the users' habitual container access pattern. The weighting coefficients λ_1 and λ_2 enable flexible trade-offs between instantaneous latency and historical consistency. This formulation provides an interpretable and practical approximation of user satisfaction while maintaining computational tractability for the subsequent optimization.

E. Problem Definitions

We first consider the GRAP problem in static scenarios where user query requests are known within G.

Definition 1: Given a mobile cloud-edge network $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, a set $\widehat{U}$ of users who are submitting query requests with each query uploading raw data of size $I_u + O_u$ for analyzing, and a set V_{ES} of ESs, *the GenAI replica allocation and placement problem (GRAP)* is a utility maximization problem that aims to maximize the total utility gain of query requests submitted by users in $\widehat{U}$. This is achieved by jointly determining: 1) which containers are suitable to be installed in the ESs, subject to the current utility state (identified by the utility indicator) and the resource constraints of ESs, and 2) the placement of GenAI replicas enabled by these containers in G, subject to the memory capacity constraints of ESs.

Definition 2: Given a mobile cloud-edge network $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, a finite set of checkpoints P, a set U of users moving within G during P, a set $\widehat{U}$ of users who submit query requests with each query uploading raw data of size $I_u + O_u$ for analyzing, and a set V_{ES} of ESs, *the online GenAI replica allocation and placement problem* is an online utility maximization problem. Without any knowledge of future user mobility or query arrivals, the objective is to maximize the cumulative utility gain of query requests submitted by users in $\widehat{U}$ over all checkpoints, by dynamically determining: (1) which containers need to be installed in ESs, subject to the current utility state and resource constraints, and (2) the placement of GenAI replicas enabled by these containers in G, subject to the CPU and memory capacity constraints of ESs.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

III. Approximation Algorithm for GRAP Problem

In this section, we present an Integer Linear Program (ILP) formulation for solving the GRAP problem in small-scale instances. For large-scale instances, we propose a greedy approximation algorithm as an efficient alternative.

A. ILP Formulation

Let $\mathbf{X} = (\tilde{x}_{e,c}\tilde{h}_{e,c},\ \tilde{x}_{e,c}(1-\tilde{h}_{e,c}),\ 1-\tilde{x}_{e,c},1)$ and $\mathbf{Y} = (\lambda_1(1-\tilde{x}_{e,c}),\ \lambda_2\tilde{x}_{e,c}(1-\tilde{h}_{e,c}))$ be two binary decision vectors, where $\tilde{x}_{e,c}\tilde{h}_{e,c}=1$ means the container c is instantiated on ES e, and $\tilde{x}_{e,c}(1-\tilde{h}_{e,c})=1$ indicates c is allocated on e but not instantiated, $1-\tilde{x}_{e,c}=1$ means c is not allocated on e. Especially, if the query of user u is processed in ES $e \in V_{ES}$ with $\tilde{x}_{e,c}\tilde{h}_{e,c}=1$, the query is assigned to a GenAI replica enabled by a container in ES e. Otherwise (the query of user u is processed in cloud v_{sc} with $\tilde{x}_{e,c}\tilde{h}_{e,c}=0$), the query is processed by the primary GenAI in cloud.

The ILP formulation of the GRAP problem is defined as:

$$\max\left\{-\sum_{u\in V_{MD}}\mathbf{X}D(u,e,v_{sc})-\sum_{e=1}^{n}\sum_{c=1}^{m}\mathbf{Y}Reg(e,c)\right\}, \tag{21}$$

subject to :

(13), (18),

$$\sum_{c=1}^{m} m_c\tilde{x}_{e,c} \le m_e, \forall e, \tag{22}$$

$$\sum_{c=1}^{m} C_c\tilde{x}_{e,c}\tilde{h}_{e,c} \le C_e, \forall e, \tag{23}$$

$$\lambda_1 > \lambda_2, \tag{24}$$

$$\tilde{h}_{e,c} \le \tilde{x}_{e,c}, \quad \tilde{x}_{e,c}, \tilde{h}_{e,c} \in \{0,1\}, \forall e, c. \tag{25}$$

Objective (21) is the total utility gain via allocating and placing GenAI replicas for users V_{MD} in ESs. Constraints (22) and (23) refer to the memory and GPU resource constraints of ESs, respectively. Constraint (25) establishes the coupling relationship between GenAI replicas' placement and allocation.

B. Approximation Algorithm

To solve the GRAP problem, we transform it into a generalized assignment problem (GAP), and then leverage the existing approximation solution [28] of GAP as the approximate algorithm specifically for GRAP. Each ES e in V_{ES} is viewed as a "bin" b_e, which has a volume capacity limitation m_e (i.e., the available storage resources of the ES) and a weight capacity limitation C_e (i.e., the available CPU resources of the ES). Meanwhile, the remote cloud v is also treated as a "bin" b_{sc}, whose volume and weight capacities are both unlimited. For each type of container on an ES, we regard it as an "item" c, with the volume m_c (the storage space consumed by deploying the container) and the weight C_c (the CPU consumption of placing the container). Assigning the item c to bin b_v (where $v \in V_{ES} \cup \{v\}$) yields a profit $p_{v,c}$, i.e., the utility value obtained according to (21).

Algorithm 1: Approximation Algorithm for the GRAP Problem.

Input: A cloud-edge network G, set of ESs $V_{ES} \subseteq V$, set of users submitting queries $U_p \in V_{MD}$ with each query uploading the size $I_u + O_u$ of raw data for analyzing, storage and CPU capacities m_e, C_e for each $e \in V_{ES}$, storage and CPU requirements m_c, C_c for container c.

Output: The total utility gain of users by allocating and placing GenAI replicas in ESs V_{ES}.

1: Compute the communication latency of the shortest path between each pair of ESs.
2: Construct an item c with the volume of m_c and the weight of C_c for each container, and a bin b_e for each ES e with the volume of m_e and the weight of C_e, as well as a bin b_{sc} for the cloud v_{sc} with unlimited capacity.
3: Assign item c to a bin b_e with $v \in V_{ES} \cup v_{sc}$ brings a profit $p_{v,c}$ defined by (15) and (21).
4: Obtain set^a and set^p via the approximation algorithm.
5: **return** Solution sets set^a and set^p to the GRAP problem.

Algorithm 1 outlines the offline solver for GRAP, which generates two intermediate sets: the *allocation set* (set^a) and the *placement set* (set^p). set^a determines which containers are assigned from the cloud to edge servers, while set^p specifies how they are instantiated or reused on each node. This algorithm runs periodically as a global maintenance phase, executing full GenAI reallocation for system-wide optimization. Although container redeployment may introduce slight synchronization delay, warm-start reuse minimizes its impact.

C. Algorithm Analysis

Theorem 1. (NP-hardness): The GRAP problem in an mobile cloud-edge network $G = (V_{ES} \cup \{v\}, V_{MD}, L_{ed})$ is NP-hard.

Proof: We prove the NP-hardness of the GRAP problem via a reduction from the canonical NP-hard GAP problem [28]. As detailed in Section III-B, the GRAP problem can be reduced to the NP-hard GAP problem, demonstrating their equivalence in computational complexity. Consequently, GRAP is inherently NP-hard. □

Theorem 2. (Approximation ratio): Given $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, a single checkpoint p, and a set of users V_{MD} submitting queries at checkpoint p, there exists an approximation algorithm (Algorithm 1) for the GRAP problem. This algorithm provides an approximate solution with an approximation ratio of $\frac{1}{2+\epsilon}$, where ϵ is a constant with $0 < \epsilon \le 1$.

Proof: With the polynomial-time complexity rigorously analyzed through the discrete polynomial optimization framework [28], we can conclude that the solution achieved by Algorithm 1 is guaranteed to be at least $1/(2+\epsilon)$ times of the optimal solution. ϵ (the *granularity parameter*) is a small positive constant that controls the precision of the approximation process

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

in the underlying GAP solver. A smaller value of ϵ leads to a tighter approximation ratio, but at the cost of higher computational complexity due to finer partitioning in the rounding phase. Typically, ϵ is selected within (0,1] to balance accuracy and efficiency in practice. □

Theorem 3. (Time complexity): The time complexity of Algorithm 1 is $O(n^3 + 2n^2 \cdot m \cdot log\frac{1}{\varepsilon} + \frac{2n}{\varepsilon^4})$.

Proof: Finding the shortest paths between each pair of ESs requires $O(n^3)$ time. The allocation involves $n \cdot m$ items, while the placement requires the total number of items satisfying $\sum_{e \in V_{ES}} \frac{m_e}{s_c} \leq n \cdot m$, and the approximation algorithm in [28] is applied $O(2n^2 \cdot m \cdot log\frac{1}{\varepsilon} + \frac{2n+2}{\varepsilon^4})$ times. Consequently, the time complexity of Algorithm 1 is $O(n^3 + 2n^2 \cdot m \cdot log\frac{1}{\varepsilon} + \frac{2n}{\varepsilon^4})$. □

IV. Mobility-Aware Online Algorithm for Container Allocation

In this section, we investigate the dynamic GRAP problem characterized by evolving the user set U_p and their allocation and placement locations across successive checkpoints p. To address the prohibitive dynamic latency penalties incurred by global reconfiguration in rapidly evolving networks, we propose a mobility-aware online algorithm. We specifically investigate the special case of heterogeneous network degradation to derive a fine-grained triggering mechanism that effectively balances utility gains against migration overhead.

A. ILP Formulation

In the online setting, we model each dynamic checkpoint p as a decision point that triggers reallocation and replacement of system. Formally, we define a checkpoint as:

$$p\left(\widehat{T}, \sum_{t\in\widehat{T}}\sum_{u\in U_p} Q(u,t)\right) = \gamma \cdot \widehat{T} \cdot \sum_{t\in\widehat{T}}\sum_{u\in U_p} Q(u,t), \quad (26)$$

where $Q(u,t)$ denotes that user u issued a request at some time point t in the time horizon $\widehat{T}$, $\sum_{t\in\widehat{T}}\sum_{u\in U_p} Q(u,t)$ is the total number of query requests issued by all users in U_p within the time horizon $\widehat{T}$, and γ is the weighting coefficient for adjusting the span between checkpoints. The offline version of the dynamic GRAP problem is formulated as an ILP model. We extend the indicator variables $x_{e,c}$, $h_{e,c}$, $\tilde{x}_{e,c}$, and $\tilde{h}_{e,c}$ from the approximation algorithm to the dynamic setting. Specifically, $\tilde{x}_{e,c,p}$ indicates whether container c is allocated to ES e at checkpoint p, and $x_{e,c,p}$ represents the deployment state of container c on ES e at checkpoint p. Similarly, $\tilde{h}_{e,c,p}$ denotes whether container c is placed on ES e at checkpoint p, and $h_{e,c,p}$ represents the placement state of container c on ES e at checkpoint p.

For the convenience of presentation, we define the binary decision vectors as:

$$\begin{cases} \mathbf{X}^p = \left(\tilde{x}_{e,c,p}\tilde{h}_{e,c,p},\ \tilde{x}_{e,c,p}(1-\tilde{h}_{e,c,p}),\ 1-\tilde{x}_{e,c,p}, 1\right), \\ \mathbf{Y}^p = \left(\lambda_1(1-\tilde{x}_{e,c,p}),\ \lambda_2\tilde{x}_{e,c,p}(1-\tilde{h}_{e,c,p})\right), \\ \mathbf{x}^p = \left(x_{e,c,p}h_{e,c,p},\ x_{e,c,p}(1-h_{e,c,p}),\ 1-x_{e,c,p}, 1\right). \end{cases}$$

Note that the offline GenAI replication allocation and placement should be performed prior to checkpoint 1 for initialization. Until reaching checkpoint 1, each GenAI replica inevitably experiences allocation delays, while some replicas incur additional allocate and placement latency. To reduce latency overhead at each checkpoint for $p \geq 1$, it is crucial to determine whether existing GenAI replica need to be reused. It is necessary to evaluate the container reuse strategy by considering latency variations in conjunction with historical container usage. Based on ILP (21) for the offline GRAP problem, the ILP of the dynamic GRAP problem is formulated as:

$$\max \left\{ -\sum_{p=1}^{P}\sum_{u\in V_{MD}} \left[\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc})\right] - \sum_{p=1}^{P}\sum_{e=1}^{n}\sum_{c=1}^{m} \mathbf{Y}^p Reg(e,c) \right\}, \quad (27)$$

subject to : (13), (18),

$$\sum_{c=1}^{m} m_c \tilde{x}_{e,c,p} \leq m_e, \forall e \quad (28)$$

$$\sum_{c=1}^{m} C_c \tilde{x}_{e,c,p}\tilde{h}_{e,c,p} \leq C_e, \forall e \quad (29)$$

$$\lambda_1 > \lambda_2 \quad (30)$$

$$\tilde{h}_{e,c,p} \leq \tilde{x}_{e,c,p}, \quad \tilde{x}_{e,c,p}, \tilde{h}_{e,c,p} \in \{0,1\}, \forall e,c,t. \quad (31)$$

The objective (27) is the total utility gain via allocating and placing GenAI replicas for users V_{MD} in ESs in all checkpoints P, and we use latency difference to highlight gains in latency utility ($\mathbf{x}^0 D(u,e,v_{sc}) = 0$). The constraints (28) and (29) refer to the memory and CPU resource constraints of ESs at every checkpoint p, respectively. Constraint (31) refers to the coupling relationship between GenAI replicas' placement and allocation.

B. Online Algorithm for a Special Case of Dynamic GRAP Problem

A critical special case in the dynamic GRAP problem occurs when some ESs maintain normal utility, while others suffer from reduced system-wide utility due to performance limitations. In this case, global adjustments yield diminishing returns for high-performing ESs while introducing non-trivial latency penalties.

From (27), we can obtain the utility of each user u at current checkpoint p:

$$ut(e,p) = -\sum_{c=1}^{m} \mathbf{Y}^p Reg(e,c) - \sum_{u\in V_{MD}^e} \left[\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc})\right]. \quad (32)$$

The mean absolute deviation (MAD) of (32) is then computed based on the utility obtained by each user. If $MAD =$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Online Algorithm for the Special Case of the GRAP Problem.

Input: A cloud-edge network G, set of ESs $V_{ES} \subseteq V$, set of users submitting queries $U_p \in V_{MD}$ at each checkpoint $p \in P$ with each query uploading the size $I_u + O_u$ of raw data for analyzing, storage and CPU capacities m_e, C_e for each $e \in V_{ES}$, storage and CPU requirements m_c, C_c for container c.

Output: The overall utility of all users throughout the all checkpoints P.

1: Obtain sets set_p^a and set_p^p of GenAI replicas by invoking Algorithm 1 and allocate and place GenAI replicas in set_0^a and set_0^p to ESs at checkpoint 0;
2: $p \leftarrow 1, p' \leftarrow 0$;
3: **while** $p \leq |P|$ **do**
4: Calculate each $ut(e,p)$ and update U_p and $\overline{MAD}$;
5: Obtain set_p^a and set_p^p by invoking Algorithm 1;
6: Calculate incurred latency $\mathbf{X}^p D(u,e,v_{sc})$ and historical usage $\mathbf{Y}^p Reg(e,c)$ by set_p^a and set_p^p at checkpoint p;
7: **if** (34) is satisfied **then**
8: Allocate and place GenAI replicas with set_p^a and set_p^p;
9: $p' \leftarrow p$;
10: **end if**
11: $p \leftarrow p+1$;
12: **end while**

$\frac{1}{n}\sum_{e=1}^{n} |ut(e,p) - \overline{ut(e,p)}|$ is larger than $\overline{MAD}$, the system enters such special case at this checkpoint, where the users specified by K utility-worst ESs are selected to update the set U_p.

A natural solution to deal with above special case is to invoke Algorithm 1 at each checkpoint p in a greedy manner, which the goal of maximizing the cumulative utility. However, this inevitably leads to frequent changes of GenAI replicas, increasing cold-start latency of containers. To resolve above, we need to exploit the container reuse, i.e., determining whether to reuse existing containers at each checkpoint p. Let $D_s(p-1,p)$ denote the total static latency difference for all user queries between checkpoint $p-1$ and p. Consistent with (27), when $p=1$, $D_s(p-1,p)$ represents the total static latency of the system, i.e., $\mathbf{X}^1 D(u,e,v_{sc})$. Similarly, let $D_d(p-1,p)$ denote the total dynamic latency difference experienced by users in U_p between checkpoints $p-1$ and p, which results from the latency overhead introduced by GenAI replica adjustments. Since no GenAI replica reallocation or replacement occurs at checkpoint 1, we define $D_d(0,1)=0$. Here, we consider the change of latency difference at each checkpoint p as utility gain, which is defined as:

$$UG = \sum_{u\in U_p} \left(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc})\right) - D_d(p-1,p) - D_s(p-1,p). \tag{33}$$

We propose a special online algorithm (Algorithm 2) to address this special problem by determining how to reallocate and replace GenAI replicas at each checkpoint $p \geq 1$. At checkpoint 0, we invoke Algorithm 1 to obtain two sets set_p^a and set_p^p of GenAI replicas for serving users in U_p, allocate replicas in set_p^a and then instantiate replicas in set_p^p on ESs. At each checkpoint $p \geq 1$, we calculate each $ut(e,p)$, check if the special case exists, and update U_p and $\overline{MAD}$. Then, we try to invoke Algorithm 1 to obtain set_p^a and set_p^p for GenAI replica allocation and placement. However, above sets cannot be directly used for reallocation and replacement, and thus we design an efficient strategy to decide whether adjustments are needed at each checkpoint, detailed as below:

Suppose p' represents the last checkpoint when GenAI replicas were replaced, with an initial value of $p'=1$. Let $\alpha > 1$ be a tuning parameter. If the total dynamic latency of queries at the current checkpoint p satisfies:

$$D_d(p-1,p) - \sum_{e=1}^{n}\sum_{c=1}^{m} \mathbf{Y}^p Reg(e,c) \leq \frac{1}{\alpha} \cdot \sum_{p''=p'}^{p-1} \Bigg[\sum_{u\in U_p} \left(\mathbf{X}^{p''} D(u,e,v_{sc}) - \mathbf{x}^{p''-1} D(u,e,v_{sc})\right) - D_s(p''-1,p'')\Bigg]. \tag{34}$$

Next, we adopt set_p^a and set_p^p for replicas reallocation and replacement at checkpoint p, respectively. The parameter α represents the tolerance for delays from reallocation and replacement, while a smaller α allows more frequent allocation and placement. Then, we generalize the online algorithm (Algorithm 3) to handle arbitrary dynamic GRAP instances. Let $\Delta\mathcal{U}_p \subseteq U_{p-1} \cap U_p$ denote the set of users who submit queries at both checkpoint $p-1$ and checkpoint p, where $p \geq 1$. At checkpoint p, the necessary containers are placed to instantiate GenAI replicas for users in this set. Thus, at checkpoint $p \geq 1$, we need to determine whether to reuse these existing containers to instantiate GenAI replicas for users in $\Delta\mathcal{U}_p$, while for users in $U_p \setminus \Delta\mathcal{U}_p$, we should consider instantiating new containers at checkpoint $p \geq 1$ to allocate and place GenAI replicas for serving them. The dynamic GRAP is then solved by allocating and placing GenAI replicas for users in both $\Delta\mathcal{U}_p$ and $U_p \setminus \Delta\mathcal{U}_p$, where each checkpoint is treated as an iteration.

At checkpoint 0, the allocation set set_0^a and the placement set set_0^p for serving users in U_0 are obtained by invoking Algorithm 1, and the GenAI replicas in set_0^a and set_0^p are allocated and placed to the corresponding ESs. For each checkpoint $p \geq 1$, we update the set $\Delta\mathcal{U}_p$, and then the solution sets $set_p^a(\Delta\mathcal{U}_p)$ and $set_p^p(\Delta\mathcal{U}_p)$ are obtained by Algorithm 1.

Similarly to Algorithm 2, we adopt the following online algorithm (as shown in Algorithm 3) to determine whether to replace GenAI replicas for users in $\Delta\mathcal{U}_p$. xFirst, The cold-start latency associated with reallocating and replacing GenAI replicas for users in $\Delta\mathcal{U}_p$ is evaluated based on the total static delay difference $D_s(\Delta\mathcal{U}_p, p-1, p)$, the total dynamic delay difference $D_d(\Delta\mathcal{U}_p, p-1, p)$, and the historical usage of previous allocations and placements. At checkpoint p, if the total dynamic delay

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Online Algorithm for the GRAP Problem.

Input: A cloud-edge network G, set of ESs $V_{ES} \subseteq V$, set of users submitting queries $U_p \in V_{MD}$ at each checkpoint $p \in P$ with each query uploading the size $I_u + O_u$ of raw data, storage and CPU capacities m_e, C_e for each $e \in V_{ES}$, storage and CPU requirements m_c, C_c for container c.

Output: The overall utility of all users throughout all checkpoints P.

Obtain sets set_p^a and set_p^p of GenAI replicas by invoking Algorithm 1 and allocate and place GenAI replicas in set_0^a and set_0^p to ESs at checkpoint 0;
$p \leftarrow 1$
while $p \leq |P|$ **do**
  Identify the set of users $\Delta\mathcal{U}_p \subseteq U_{p-1} \cap U_p$, where each $u \in \Delta\mathcal{U}_p$ issues queries in checkpoints $p-1$ and p;
  Obtain the sets $set_p^a(\Delta\mathcal{U}_p)$ and $set_p^p(\Delta\mathcal{U}_p)$ of GenAI replicas by invoking Algorithm 1;
  Identify the incurred dynamic utility $D_d(\Delta\mathcal{U}_p, p) - \sum_{e=1}^{n}\sum_{c=1}^{m} \mathbf{Y}^p Reg(e,c)$ by $set_p^a(\Delta\mathcal{U}_p)$ and $set_p^p(\Delta\mathcal{U}_p)$ at checkpoint p;
  if (35) is satisfied **then**
    Adopt $set_p^a(\Delta\mathcal{U}_p)$ and $set_p^p(\Delta\mathcal{U}_p)$ for reallocating and replacing the GenAI replicas for users in $\Delta\mathcal{U}_p$;
  end if
  Update the storage and CPU capacities m_e, C_e for each $e \in V_{ES}$
  Invoke Algorithm 1 to identify $set_p^a(U_p \backslash \Delta\mathcal{U}_p)$ and $set_p^p(U_p \backslash \Delta\mathcal{U}_p)$ of GenAI replicas in ESs with updated resources for users in $U_p \backslash \Delta\mathcal{U}_p$;
  $p \leftarrow p+1$;
end while

difference is no greater than $1/\beta$ times the total query utility gain without considering dynamic delays (35), then set_p^a and set_p^p are used to reallocate and replace GenAI replicas on ESs for users in $\Delta\mathcal{U}_p$. Otherwise, no adjustment is made to the GenAI replicas, and the allocation and placement strategy from checkpoint $p-1$ is retained for serving users in $\Delta\mathcal{U}_p$.

$$D_d(\Delta\mathcal{U}_p, p-1, p) - \sum_{e=1}^{n}\sum_{c=1}^{m} \mathbf{Y}^p Reg(e,c) \leq \frac{1}{\beta} \cdot \left[\sum_{u \in \Delta\mathcal{U}_p} \left(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc}) \right) - D_s(\Delta\mathcal{U}_p, p-1, p) \right]. \quad (35)$$

After allocating and placing GenAI replicas for users in $\Delta\mathcal{U}_p$, the residual storage and computing capacities of ESs are updated. Then, Algorithm 1 is invoked to compute $set_p^a(U_p \setminus \Delta\mathcal{U}_p)$ and $set_p^p(U_p \setminus \Delta\mathcal{U}_p)$ and allocating and placing GenAI replicas on ESs serve for users in $U_p \setminus \Delta\mathcal{U}_p$.

C. Algorithm Analysis

Lemma 1. (Latency upper bound): According to Algorithm 2, the cumulative dynamic delay of user queries is no greater than $1/\alpha$ times the cumulative utility gain across all checkpoint $|P|$.

Proof: Based on the update decision criterion, we formulate the cumulative dynamic delay across all checkpoints to establish the latency upper bound; for detailed definitions of update checkpoints $\hat{p}_k$ and the complete mathematical derivation, please refer to *Supplementary Material Section S1*.

Lemma 2. (Upper bound of the utility gain): For the dynamic GRAP special case, the optimal solution's utility (UG^*) of Algorithm 2 is upper bounded by no more than a factor of σ (multiplied by the cumulative utility of the static latency computed by Algorithm 1), where $\sigma = \max_{p \in P} \left[\frac{\max_{e \in V_{ES}} |ut(e,p) - \overline{ut(e,p)}|}{\min_{e \in V_{ES}} |ut(e,p) - \overline{ut(e,p)}|} \right]$ also means the maximum proportion of the difference in utilities of different ESs under any checkpoint p.

Proof: The proof derives the upper bound by analyzing the heterogeneity factor σ, which limits the utility difference ratio between the optimal solution and the baseline across all edge servers. please refer to *Supplementary Material Section S1*.

Theorem 4. (Competitive ratio): Given $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, all of checkpoints P, and a set of users U_p submitting queries at checkpoint p, there exists an approximation algorithm (Algorithm 2) for the GRAP problem with competitive ratio of $\frac{1}{\sigma}(1 - \frac{1}{\alpha})$.

$$\begin{aligned} UG &= \sum_{p=1}^{|P|} \sum_{u \in U_p} \left(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc}) \right) \\ &\quad - \sum_{p=1}^{|P|} D_d(p-1,p) - \sum_{p=1}^{|P|} D_s(p-1,p) \\ &\geq \sum_{p=1}^{|P|} \sum_{u \in U_p} \left(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc}) \right) \\ &\quad - \sum_{p=1}^{|P|} D_s(p-1,p) - \frac{1}{\alpha} \cdot \sum_{p=1}^{|P|} \Big(\sum_{u \in U_p} \Big(\mathbf{X}^p D(u,e,v_{sc}) \\ &\quad - \mathbf{x}^{p-1} D(u,e,v_{sc}) \Big) - D_s(p-1,p) \Big) \\ &\geq \left(1 - \frac{1}{\alpha}\right) \sum_{p=1}^{|P|} \Big(\sum_{u \in U_p} \left(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1} D(u,e,v_{sc}) \right) \\ &\geq \frac{1}{\sigma} \left(1 - \frac{1}{\alpha}\right) \cdot UG. \end{aligned} \quad (36)$$

Proof: Given that UG and UG represent the cumulative utility gains achieved by the optimal solution and by the solution produced by Algorithm 2, respectively, for the special case of the dynamic GRAP problem. Then, as (36), the theorem is proven.□

Theorem 5. (Time complexity of the special case): The time complexity of Algorithm 2 is $O(n^3 + |P| \cdot 2n^2 \cdot m \cdot log\frac{1}{\varepsilon} + |P| \cdot \frac{2n}{\varepsilon^4} + |P| \cdot m \cdot n)$.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

Proof: Since the time complexity of Algorithm 1 is $O(n^3 + 2n^2 \cdot m \cdot \log \frac{1}{\varepsilon} + \frac{2n}{\varepsilon^4})$, where n^3 corresponds to the time complexity of computing the shortest paths between each pair of nodes, which is performed once across all checkpoints. At each checkpoint p, the utility for each ES should be calculated to partition the user set U_p. Consequently, the time complexity of Algorithm 2 is $O(n^3 + |P| \cdot 2n^2 \cdot m \cdot \log \frac{1}{\varepsilon} + |P| \cdot \frac{2n}{\varepsilon^4} + |P| \cdot m \cdot n)$. □

Theorem 6. (Time complexity of Algorithm 3): Given $G = (V_{ES} \cup \{v_{sc}\}, V_{MD}, L_{ed})$, over all checkpoints P, and a set of users U_p submitting queries at checkpoint p, a feasible online solution is delivered by Algorithm 3 for the dynamic GRAP problem. And the time complexity of Algorithm 3 is $O(n^3 + |P| \cdot 2n^2 \cdot m \cdot log\frac{1}{\varepsilon} + |P| \cdot \frac{2n}{\varepsilon^4})$.

Proof: Since Algorithm 3 does not result in any resource violations, it ensures a feasible solution to the dynamic GRAP problem. The time complexity of Algorithm 3 is primarily determined by the invoking of Algorithm 1 for the user sets $\Delta\mathcal{U}_p$ and $U_p \setminus \Delta\mathcal{U}_p$ at each checkpoint. Given that $\Delta\mathcal{U}_p \in V_{MD}$ and $U_p \setminus \Delta\mathcal{U}_p \in V_{MD}$, based on Theorem 2, we can get that the time complexity of Algorithm 3 is $O(n^3 + |P| \cdot 2n^2 \cdot m \cdot \log \frac{1}{\varepsilon} + |P| \cdot \frac{2n}{\varepsilon^4})$. □

Lemma 3. (Latency upper bound): For the dynamic GRAP (Algorithm 3), let $\Delta U_p \subseteq U_{p-1} \cap U_p$ be the users issuing queries at both checkpoints $p-1$ and p. Under the triggering rule in (35) with $\beta > 1$, the cumulative dynamic latency satisfies:

$$\sum_{p=1}^{|P|}\Big[D_d(\Delta U_p, p-1, p) - \sum_{e,c} \mathbf{Y}^p Reg(e,c)\Big] \leq \frac{1}{\beta}\sum_{p=1}^{|P|}\Big(\sum_{u\in\Delta U_p}(\mathbf{X}^p - \mathbf{x}^{p-1})D(u,e,v_{sc}) - D_s(\Delta U_p, p-1, p)\Big). \tag{37}$$

Since users in $U_p \setminus \Delta U_p$ are newly scheduled and incur no carry-over overhead, the total dynamic latency over all users obeys:

$$\sum_{p=1}^{|P|}\Big[D_d(p-1,p) - \sum_{e,c} \mathbf{Y}^p Reg(e,c)\Big] \leq \frac{1}{\beta}\sum_{p=1}^{|P|}\Big(\sum_{u\in U_p}(\mathbf{X}^p - \mathbf{x}^{p-1})D(u,e,v_{sc}) - D_s(p-1,p)\Big). \tag{38}$$

Proof: At each checkpoint p, Algorithm 3 identifies ΔU_p and calls Algorithm 1 to obtain (set^a, set^p) for reallocation. By (35), the dynamic latency caused by reallocation of ΔU_p is upper-bounded by $1/\beta$ of its static utility gain, yielding (37). Users in $U_p \setminus \Delta U_p$ are new arrivals with no inherited delay; summing both parts over all p gives (38). □

Lemma 4. (Upper bound of the optimal utility): Let UG^* denote the optimal cumulative utility gain of the general dynamic GRAP over all checkpoints. With the heterogeneity factor σ defined in (39), we have

$$UG^* \leq \sigma\Bigg[\sum_{p=1}^{|P|}\sum_{u\in U_p}\big(\mathbf{X}^p D(u,e,v_{sc}) - \mathbf{x}^{p-1}D(u,e,v_{sc})\big) - \sum_{p=1}^{|P|} D_s(p-1,p)\Bigg]. \tag{39}$$

Proof: The proof follows the same reasoning as that of Lemma 2 for Algorithm 2. Please refer to *Supplementary Material Section S1* □

V. Experimental Evaluation

In this section, we assess the algorithm's performance for the GRAP problem at each checkpoint as well as for the dynamic GRAP problem across all checkpoints.

A. Experimental Settings

For the cloud-edge network, simulated by the GT-ITM tool [38], the count of ESs varies from 50 to 250, with each ES paired with a group of users. For each ES, the allocation of storage space is randomly selected within [60 GB, 180 GB], while the CPU resources are allocated within [6 GB, 18 GB]. To instantiate a GenAI replica, the container requires storage space within [2 GB, 50 GB], with memory consumption within [1 GB, 6 GB] [39]. For the network comprising 2000 users, the checkpoint size is determined by tuning the parameter γ, such that 1000 requests within 5 seconds can be aggregated as one checkpoint. Users can move to a random location between consecutive checkpoints. The size of the data uploaded by a user query is set within [100 MB, 500 MB] [40]. The cold start delay of a container is set within [100 ms, 500 ms] [43]. The processing speed of query data is configured within [0.5 MB, 2 MB] perms, while the remote cloud operates at a fixed rate of 20 MB perms [44]. The communication delay of a unit of data (1MB) between each pair of ESs is set within [0.2 ms, 1 ms] [45]. Each data point in the figures corresponds to the average performance derived from 30 independent MEC instances with identical network scale.

We set α to three different scenarios: *low deviation* ($\sigma \in [1,3]$), where the performance gap among edge servers is minimal; *moderate deviation* ($\sigma \in (3,7]$), where some edge servers perform worse; and *high deviation* ($\sigma > 7$), where the performance of edge servers becomes severely imbalanced. In all experiments, we set $\beta = 3$ to evaluate its impact on the algorithm's performance.

We evaluate the proposed approach, Algorithm 1 (abbreviated as Algorithm 1), against the following three baseline algorithms for solving the GRAP problem.

1) *GDY:* Each user query is assigned to a container that's greedily instantiated on an ES. If that fails, the query executes in the remote cloud.
2) *LP:* The relaxed linear program (LP) solution (21) is obtained by relaxing the binary decision variables $\tilde{x}_{e,c}$

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

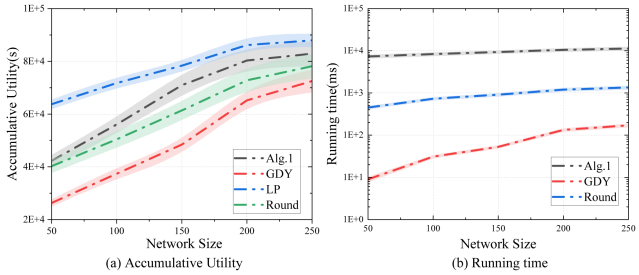


Fig. 2. Algorithm performance for the GRAP problem.

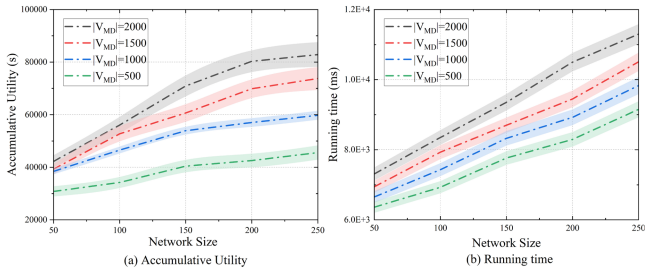


Fig. 3. Performance of Algorithm 1 with the varied number of users' queries.

and $\tilde{h}_{e,c}$ of the ILP formulation for the GRAP problem to continuous variables within the interval [0,1].

3) *Round:* A variant of the [42] rounding algorithm. We random-round the LP solution (21) into an integer solution, then iteratively remove random GenAI replicas from any overloaded ES until all constraints are met.

We evaluate Algorithm 2 (denoted as Algorithm 2) designed for the specialized GRAP problem against three benchmark algorithms: GDY_d, LP_d, and $Round_d$. Specifically, algorithms GDY_d and $Round_d$ repeatedly invoke algorithms GDY and $Round$, respectively, to allocate and place GenAI replicas at each checkpoint. Algorithm LP_d corresponds to the relaxed linear program solution (27) for the offline dynamic GRAP problem and serves as an upper bound on its optimal solution.

Moreover, we also assess Algorithm 3 (denoted as Algorithm 3), proposed for the dynamic GRAP problem, against the same benchmark algorithms: GDY_d, LP_d, and $Round_d$.

B. Algorithm Performance for the GRAP Problem

We first evaluate the performance of our Algorithm 1 against three baselines by varying the network size-gradually increasing the number of ES from 50 to 250 and comparing them along two metrics: cumulative utility and running time. The cumulative utility is quantified in accordance with (32) as the aggregate latency reduction obtained by offloading user queries to ESs, relative to a baseline in which all queries are executed solely in the cloud. Fig. 2 presents the comparisons of cumulative utility and running time for the four algorithms under different network sizes, while the LP baseline is omitted from the runtime comparison since it only provides a relaxed upper-bound reference rather than a deployable discrete algorithm.

Fig. 2(a) shows that the cumulative utilities of the four algorithms rank in descending order as LP, Algorithm 1, GDY, and $Round$. In particular, at a network size of 250 ESs, Algorithm 1 attains a 14.28% higher cumulative utility than GDY and a 6.06% improvement over $Round$. This gain derives from the strategic placement of GenAI replicas within the ESs, which enhances user satisfaction for checkpoint-based query servicing. Fig. 2(b) further reveals that, due to the invocation of the approximation algorithm from [28], Algorithm 1 incurs a longer running time than GDY.

Subsequently, we analyze the effect of the number of query requests between two checkpoints on both cumulative utility and runtime. Since the capacity of each checkpoint interval to accommodate requests is governed by the parameter γ, we decrease γ to observe its impact. Fig. 3 presents the performance curves of Algorithm 1 under four request-load settings: $|V_{MD}| =$ 500, 1000, 1500, and 2000. As shown in Fig. 3(a), with the network size fixed at 250 ESs, handling 2000 concurrent user requests yields a cumulative utility that is 122% higher than that achieved with 500 requests, demonstrating that our algorithm attains greater utility under heavier loads. Moreover, Fig. 3(b) indicates that increasing the number of requesting users leads to a corresponding growth in the algorithm's runtime.

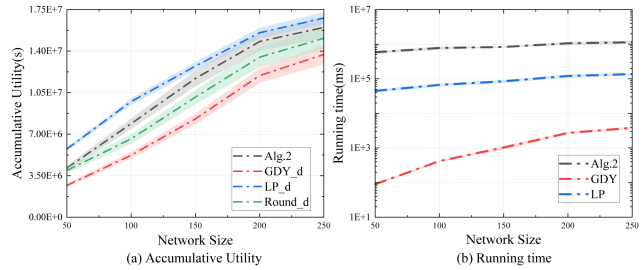


Fig. 4. Performance for the special dynamic GRAP problem.

C. Algorithm Performance for the Special Dynamic GRAP and Dynamic GRAP Problems

We compare Algorithm 2 with three baselines (GDY_d, LP_d, and $Round_d$) in cumulative utility and total runtime across different network sizes, as shown in Fig. 4. Algorithm 2 achieves the second-highest cumulative utility, slightly below LP_d but outperforming the other two. When the network size reaches 250, its utility exceeds GDY_d and $Round_d$ by 16.71% and 6.06%, respectively, demonstrating the effectiveness of its container reuse strategy for GenAI replica allocation across checkpoints. Fig. 4(b) shows that Algorithm 2 incurs a longer runtime than GDY_d and $Round_d$ because it repeatedly invokes Algorithm 1, exhibiting similar runtime characteristics.

We further analyze the effect of parameter α under varying network sizes and user numbers (Fig. 5). When $\alpha = 5$, the cumulative utility is 5.2%–30.4% higher than that at $\alpha = 7$, and 32.7%–49.7% higher than at $\alpha = 3$. As the number of users decreases, the optimal α tends to be smaller; with only 500 users, $\alpha = 3$ slightly outperforms $\alpha = 5$. A larger α increases allocation frequency and reduces latency tolerance, but overly

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

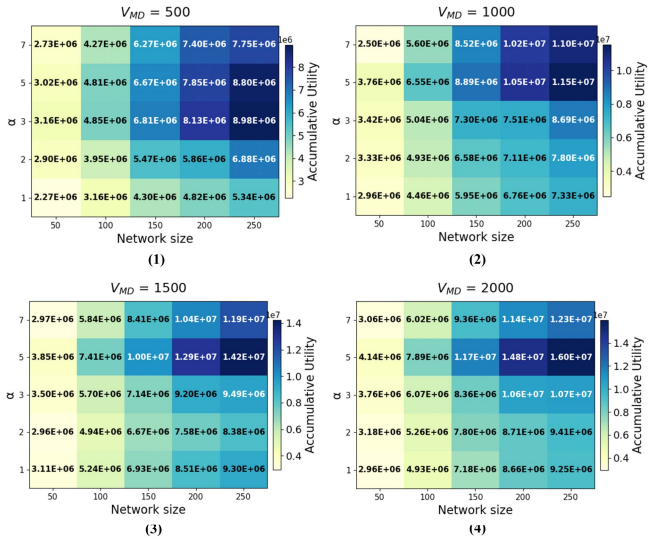


Fig. 5. Comparison of accumulative utility of different α in different network sizes with different number of users.

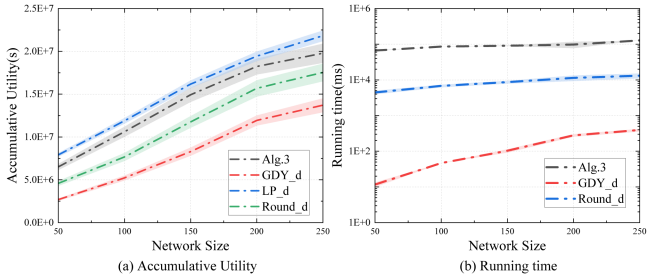


Fig. 6. Performance for the dynamic GRAP problem.

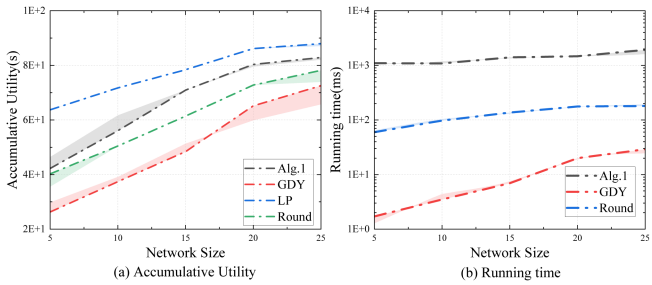


Fig. 7. Performance for the GRAP problem in a small-scale real environment.

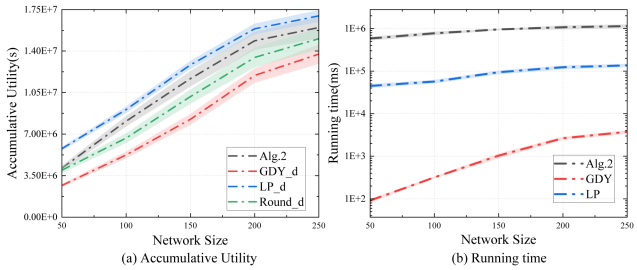


Fig. 8. Performance for the special dynamic GRAP problem in a small-scale real environment.

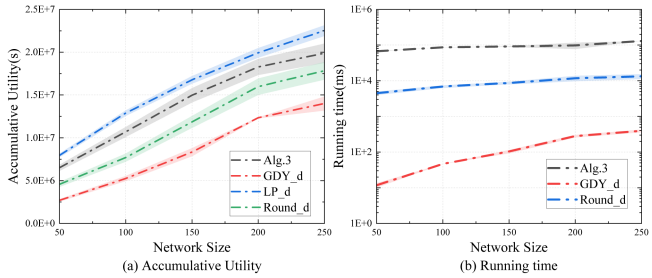


Fig. 9. Performance for the dynamic GRAP problem in a small-scale real environment.

frequent adjustments may degrade overall utility, suggesting that smaller systems benefit from lower α.

We supplemented the experimental analysis on σ by comparing the cumulative utility of Algorithm 2 and Algorithm 3 across varying network sizes and ES utility deviations (represented by different ranges of σ). The experimental results are presented in *Supplementary Material Section S2*. And to verify the robustness of Algorithm 3, we evaluated the impact of the tuning parameter β on the trade-off between system stability and responsiveness. The experimental results are provided in *Supplementary Material Section S2*.

Similarly, we perform a comparison of Algorithm 3 against the three baselines GDY_d, LP_d, and $Round_d$ in terms of cumulative utility and runtime, with the results presented in Fig. 6. The findings demonstrate that Algorithm 3 delivers strong performance: when the network size is 250, its cumulative utility exceeds that of GDY_d and $Round_d$ by 44.30% and 12.85%, respectively, and lags only LP_d by 9.32%.

To further demonstrate the superiority of the proposed method in the context of modern GenAI computing, we compare Algorithm 3 with two state-of-the-art baselines, D2SAC [31] and JMARA [37].The experimental results are provided in *Supplementary Material Section S2*.

To validate simulation results under real hardware and network conditions, we built a small cloud–edge testbed in our lab (Fig. 10). It comprised 25 Raspberry Pi-5 (8 GB) devices and one Jetson AGX Orin (64 GB) server acting as the cloud. The inter-device network maintained 100 Mbps bandwidth, and controlled latency was injected using the Linux `tc` utility to emulate large-scale environments.

We again evaluate performance along cumulative utility and total runtime with the results shown in Figs. 7, 8, and 9 for Algorithm 1, Algorithm 2, and Algorithm 3, respectively. Although the limited capacity of our hardware required us to simulate a smaller GenAI model (Llama27Bint8 and DistilGPT2), the experimental results align closely with the trends seen in simulation. This confirms that our algorithms can effectively optimize GenAI container allocation and placement even under real hardware constraints and network jitter.

While our testbed utilizes Raspberry Pi devices rather than enterprise-grade GPUs, this choice follows a ”scale-down” methodology designed to validate orchestration logic under

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

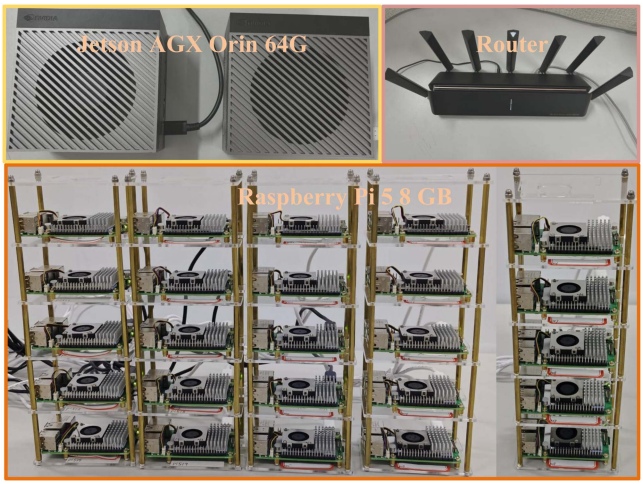


Fig. 10. Small-scale real-world experimental equipment.

strict resource constraints rather than pursuing raw inference speed. By executing quantized models (e.g., Llama-2-7B-int8) on low-power devices, we reproduce behavioral characteristics isomorphic to large-scale deployments—specifically memory saturation, cold-start sensitivity, and network dependence—thereby confirming the testbed's validity in simulating real-world contention. Notably, migrating to high-performance hardware would significantly reduce processing latency (D_{proc}), causing network transmission (D_{trans}) to dominate the total latency. This shift from "compute-bound" to "network-bound" bottlenecks would further reinforce the criticality of the network-centric scheduling strategy proposed in this work.

VI. Related Work

Recent studies have witnessed breakthroughs of generative artificial intelligence (GenAI) in various content-generative task [8], [9], [10], [16], [21], [27], [30], [32]. A series of large-scale GenAI models have been proposed and developed, e.g., MirrorGAN [9], GPT-3 (with 175 billion parameters) [16], PaLM (with 540 billion parameters) [27] and NVAE [21]. Such models require massive storage and computing resources for training and inference and thus relies on cloud computing. However, the remote cloud access incurs significant latency, also increasing network congestion [18]. Serverless mobile edge computing (MEC) can be integrated with cloud, i.e., serveless cloud-edge network, to enable the delivery of real-time GenAI services through ultra-low-latency co-location [15], where dynamic function management is excuted in edge servers via lightweight containerization (e.g., Docker) [26]. A series of container optimization strategies have been devised for serverless cloud-edge networks [17], [23], [24], [26], e.g., Skippy [15] and COSE [24].

The combination of MEC and GenAI has attracted lots of attention [31], [35], [36], [37], [41], introducing new dimensions in boosting network service. For example, H.Du et al. [31] proposed an AIGCasaService (AaaS) architecture to deploy large-scale generative models onto wireless edge networks. M.Xu et al. [35] proposed an endtoend AIGC framework to achieve lowlatency and privacypreserving realtime AIGC services. Then, a latent action diffusion scheduling framework [41] were proposed, where the AIGC service deployment is modeled as an offloading optimization problem across multiple edge nodes.

However, no existing works focus on the GenAI service provision in serverless cloud-edge networks. To address the high overhead and complex scheduling of GenAI containerization, this work aims to optimize container scheduling and replica management in both system architecture and algorithms. The proposed methods effectively improve GenAI replica orchestration efficiency by reducing service latency and increasing user utility; in particular, at a network size of 250, Algorithm 1 improves cumulative utility over GDY and Round by 14.28% and 6.06%, respectively, and Algorithm 3 improves that over GDY$_d$ and Round$_d$ by 44.30% and 12.85%, respectively.

Difference from VNF Placement: It is worth noting that the GenAI replica allocation studied in this paper fundamentally differs from traditional Virtual Network Function (VNF) placement. While VNFs are typically long-running services with static resource requirements (e.g., fixed CPU/bandwidth), serverless GenAI replicas are ephemeral and exhibit highly dynamic memory footprints due to the accumulation of KV cache. Consequently, existing VNF placement algorithms, which focus on static resource provisioning and throughput maximization, are ill-suited for addressing the cold-start latency and dynamic memory constraints (OOM risks) inherent in GenAI inference.

VII. Conclusion and Future Work

This paper presents a novel latency-sensitive Generative AI provision mechanism via dynamic utility maximization in serverless mobile cloud-edge networks. To realize this, we first propose two joint decision formulations, including GRAP and dynamic GRAP, involving 1) which containers to be prioritized for ES redeployment and maintained in warm-start mode, and 2) which containers to be offloaded or cold-started. Then, we propose an approximation algorithm for solving the GRAP problem, which can guarantee stable approximation ratio of optimal solution. Then, we design online algorithms to address the dynamical GRAP problem, accounting for mobile users with value uncertainty and stochastic request arrivals. This solution enables the cloud-edge system to determine optimal GenAI replica placement with a bounded regret. Furthermore, a rigorous theoretical analysis supports the desired properties of the proposed mechanism. Experimental results empirically validate the effectiveness of the proposed designs. In particular, Algorithm 1 improves cumulative utility over GDY and Round by 14.28% and 6.06%, respectively, under a network size of 250, while Algorithm 3 improves cumulative utility over GDY$_d$ and Round$_d$ by 44.30% and 12.85%, respectively, and further outperforms two state-of-the-art baselines, D2SAC and JMARA, as reported in Supplementary Material Section S2.

Limitation: While our algorithm reduces service latency through optimal task placement, some challenges remain for

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.

GenAI deployment, such as reliability gap (no failover mechanism for unstable edge nodes) and privacy risk (intermediate results vulnerable during inter-server transmission). These aspects constitute our primary focus for subsequent research.

REFERENCES

[1] Y.-C. Wang, J. Xue, C. Wei, and C.-C. J. Kuo, "An overview on generative AI at scale with edge–cloud computing," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 2952–2971, 2023.

[2] M. Christodorescu et al., "Securing the future of GenAI: Policy and technology," 2024, *arXiv:2407.12999*.

[3] Q. Chen, M. Yi, J. Li, N. Li, H. Gao, and Z. Cai, "Optimal and approximate parallelism-based computation offloading algorithms for real-time multimodal learning at the edge," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2025, pp. 1–10.

[4] T. Qiao, J. Zhang, D. Xu, and D. Tao, "MirrorGAN: Learning text-to-image generation by redescription," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Long Beach, CA, USA, Jun. 2019, pp. 1505–1514.

[5] E. Adamopoulou and L. Moussiades, "An overview of Chatbot technology," in *Proc. IFIP Int. Conf. Artif. Intell. Appl. Innovations*, 2020, pp. 373–383.

[6] I. Maglogiannis, L. Iliadis, and E. Pimenidis, Eds., *Artificial Intelligence Applications and Innovations* (IFIP Advances in Information and Communication Technology), vol. 584. Berlin, Germany: Springer, 2020.

[7] M. Xu et al., "Generative AI-empowered simulation for autonomous driving in vehicular mixed reality metaverses," in *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 5, pp. 1064–1079, Sep. 2023.

[8] L. Ma et al., "TBCIM: Two-level blockchain-aided edge resource allocation mechanism for federated learning service market," *IEEE/ACM Trans. Netw.*, vol. 33, no. 6, pp. 3408–3423, Dec. 2025, doi: 10.1109/TON.2025.3589017.

[9] Y. Cheng, Z. Gan, Y. Li, J. Liu, and J. Gao, "Sequential attention GAN for interactive image editing," in *Proc. 28th ACM Int. Conf. Multimedia*, Seattle, WA, USA, Oct. 2020, pp. 4383–4391.

[10] OpenAI, "GPT-4 technical report," Mar. 2023, *arXiv:2303.08774*.

[11] DeepSeek-AI, "DeepSeek-V3 technical report," 2024, *arXiv:2412.19437*.

[12] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2337, Fourthquarter 2017, doi: 10.1109/COMST.2017.2745201.

[13] Q. Chen et al., "Towards real-time inference offloading with distributed edge computing: The framework and algorithms," *IEEE Trans. Mobile Comput.*, vol. 23, no. 7, pp. 7552–7571, Jul. 2024.

[14] L. Ma, X. Wang, X. Wang, L. Wang, L. Shi, and M. Huang, "TCDA: Truthful combinatorial double auctions for mobile edge computing in industrial Internet of Things," *IEEE Trans. Mobile Comput.*, vol. 21, no. 11, pp. 4125–4138, Nov. 2022.

[15] T. Rausch, A. Rashed, and S. Dustdar, "Optimized container scheduling for data-intensive serverless edge computing," *Future Gener. Comput. Syst.*, vol. 114, pp. 259–271, 2021.

[16] T. B. Brown et al., "Language models are few-shot learners," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, Dec. 2020, pp. 1877–1901.

[17] G. Prekas, M. Kogias, and E. Bugnion, "ZygOS: Achieving low tail latency for microsecond-scale networked tasks," in *Proc. 26th Symp. Oper. Syst. Princ.*, Shanghai, China, Oct. 2017, pp. 325–341.

[18] C.-H. Hong and B. Varghese, "Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms," *ACM Comput. Surveys*, vol. 52, no. 5, pp. 1–37, 2019.

[19] H. Shafiei, A. Khonsari, and P. Mousavi, "Serverless computing: A survey of opportunities, challenges, and applications," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–32, 2022.

[20] G. McGrath and P. R. Brenner, "Serverless computing: Design, implementation, and performance," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. Workshops*, Atlanta, GA, USA, Jun. 2017, pp. 405–410.

[21] A. Vahdat and J. Kautz, "NVAE: A deep hierarchical variational autoencoder," in *Proc. 34th Conf. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, Dec. 2020, pp. 19667–19679.

[22] J. M. Hellerstein et al., "Serverless computing: One step forward, two steps back," 2018, *arXiv:1812.03651*.

[23] K. Kaffes, N. J. Yadwadkar, and C. Kozyrakis, "Centralized core-granular scheduling for serverless functions," in *Proc. ACM Symp. Cloud Comput.*, Santa Cruz, CA, USA, Nov. 2019, pp. 158–164.

[24] N. Akhtar, A. Raza, V. Ishakian, and I. Matta, "COSE: Configuring serverless functions using statistical learning," in *Proc. IEEE Int. Conf. Comput. Commun.*, Beijing, China, Apr. 2020, pp. 129–138.

[25] J. Li et al., "Mobility-aware utility maximization in digital twin-enabled serverless edge computing," *IEEE Trans. Comput.*, vol. 73, no. 7, pp. 1837–1840, Jul. 2024.

[26] I. E. Akkus et al., "{SAND}: Towards {high-performance}serverless computing," in *Proc. USENIX Annu. Tech. Conf.*, Boston, MA, USA, Jul. 2018, pp. 923–935.

[27] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, pp. 1–113, 2023.

[28] R. Cohen, L. Katzir, and D. Raz, "An efficient approximation for the generalized assignment problem," *Inf. Process. Lett.*, vol. 100, pp. 162–166, 2006.

[29] M. Xu et al., "Unleashing the power of edge-cloud generative AI in mobile networks: A survey of AIGC services," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1127–1137, Secondquarter 2024.

[30] L. Ma, S. Cheng, and Y. Shi, "Enhancing learning efficiency of brain storm optimization via orthogonal learning design," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 11, pp. 6723–6742, Nov. 2021.

[31] H. Du et al., "Diffusion-based reinforcement learning for edge-enabled AI-generated content services," in *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8902–8918, Sep. 2024.

[32] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, vol. 139, pp. 8162–8171.

[33] Y. Liang, P. Yang, Y. He, and F. Lyu, "Resource-efficient generative AI model deployment in mobile edge networks," in *Proc. IEEE Glob. Commun. Conf.*, 2024, pp. 2647–2652.

[34] L. Ma, Y. Zhou, L. Wang, X. Wang, G. Han, and C. F. Chiasserini, "Near optimal locality-aware task allocation towards stable blockchain-based MEC system: A potential game approach," *IEEE Trans. Mobile Comput.*, vol. 25, no. 5, pp. 6673–6687, May 2026, doi: 10.1109/TMC.2025.3638824.

[35] Z. Ding, D. Xu, R. Schober, and H. V. Poor, "Hybrid NOMA offloading in multi-user MEC networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 7, pp. 5377–5385, Jul. 2022, doi:10.1109/TWC.2021.3139932.

[36] S. Baraković and L. Skorin-Kapov, "Survey and challenges of QoE management issues in wireless networks," *J. Comput. Netw. Commun.*, vol. 2013, no. 1, 2013, Art. no. 165146.

[37] S. Gao et al., "Joint model assignment and resource allocation for cost-effective mobile generative services," in *Proc. IEEE Glob. Commun. Conf.*, Cape Town, South Africa, Dec. 2024, pp. 590–595.

[38] E. W. Zegura, K. L. Calvert, and S. Bhattacharjee, "Modeling topology of large internetworks," *Georgia Inst. Technol.*, GT-ITM, 1996. Accessed: May 26, 2025. [Online]. Available: http://www.cc.gatech.edu/projects/gtitm/

[39] L. Manduchi et al., "On the challenges and opportunities in generative AI," 2025, *arXiv:2403.00025*.

[40] Z. Xu, Y. Fu, Q. Xia, and H. Li, "Enabling age-aware Big Data analytics in serverless edge clouds," in *Proc. IEEE Conf. Comput. Commun.*, Piscataway, NJ, USA, IEEE, Apr. 2023, pp. 1–10.

[41] C. Xu et al., "Accelerating AIGC services with latent action diffusion scheduling in edge networks," 2024, *arXiv:2412.18212*.

[42] M. Vaezi, K. Noroozi, T. D. Todd, D. Zhao, and G. Karakostas, "Digital twin placement for minimum application request delay with data age targets," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11547–11557, Jul. 2023.

[43] J. Manner, M. Endreß, T. Heckel, and G. Wirtz, "Cold start influencing factors in function as a service," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput. Companion*, Zurich, Switzerland, Dec. 2019, pp. 181–188.

[44] J. Li et al., "Wait for fresh data? Digital twin empowered IoT services in edge computing," in *Proc. IEEE 20th Int. Conf. Mobile Ad Hoc Smart Syst.*, Piscataway, NJ, USA, IEEE, 2023, pp. 397–405.

[45] Z. Xu et al., "Energy-aware collaborative service caching in a 5G-enabled MEC with uncertain payoffs," *IEEE Trans. Commun.*, vol. 70, no. 2, pp. 1058–1071, Feb. 2022.

[46] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in *Proc. 22nd Int. Conf. Architecture Support Program. Lang. Oper. Syst.*, Xi'an, China, Apr. 2017, pp. 615–629.

[47] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proc. IEEE*, vol. 107, no. 8, pp. 1738–1762, Aug. 2019.

Authorized licensed use limited to: Vidya Vardhaka College Of Engineering Mysore. Downloaded on August 12,2026 at 06:23:29 UTC from IEEE Xplore. Restrictions apply.